Colin Fergusson, Ishimwe Hanyurwimfura
3664465
CS4745
Group Project Performance Report

Problem Definition and Inputs:

Our project implements image processing through a pipeline of filters that get applied to an image that is provided as input. The pipeline consists of a Gaussian Blur filter that reduces noise and smooths the image, a Median filter which removes impulse or salt-and-pepper noise, then a Sobel Edge Detection filter which computes image gradients to highlight edges. The filters are applied in that order and are applied to pgm image files that are either 512x512 pixels, 1920x1080 pixels, or 4096x4096 pixels. When running the program you input either "small", "medium", or "large" to choose which size image the pipeline is applied to.

Algorithm Description:

OpenMP:

The OpenMP algorithm first reads the selected input image (small, medium, or large) into an input buffer and allocates one output buffer of the same size, It then creates a single OpenMP parallel region so the same thread team is reused for the whole pipeline instead of creating new threads for every filter. Inside this region, the processing is done in three ordered stages. In the Gaussian blur stage, the image is traversed by pixel coordinates (y, x), and the nested loops are distributed among threads with OpenMP for work sharing, so each thread computes the blurred value for a different set of pixels and stores the result in output; corner, edge, and interior pixels are handled separately so boundary accesses stay valid. After that loop finishes, one thread enters an OpenMP single section to stop the timer, print the Gaussian runtime, save gaussian.pgm, and swap the input and output data pointers so that the blurred image becomes the new input. The same pattern is then repeated for the median filter where threads again divide the image pixels, each thread builds a local 3×3 window around its assigned pixel, sorts those values, selects the middle value, and writes the median-filtered result to output; once complete, a single thread records the time, saves median.pgm, and swaps the buffers again. Finally, in the Sobel stage, the threads once more split the image pixels, compute horizontal and vertical gradients using the Sobel kernels, combine them with the gradient magnitude formula, and write the edge result into output. When that stage finishes, one thread records the final runtime and saves sobel.pgm. In this design, the expensive pixel level work is parallel,

while only timing, file output, and buffer swapping remain serial, which preserves the logic of the serial algorithm while accelerating execution through shared memory parallelism.

MPI:

The MPI algorithm starts off by having rank 0 load in the image the user chose then copies the image into a buffer that can be used when applying the filters, it then broadcasts the details of the image (height, width, etc) to every other process. The first filter then gets applied to the buffer by dividing the rows of pixels between the processes and giving the remainder where available. Rank 0 then scatters the data of the image to every process based on what rows they were assigned before. Now that every process has the data they need access to the halo area is calculated between processes that have sections adjacent to each other so they don't try to access data they don't have when applying the filter. Now that every process has the data they need with the crossover regions taken into account each process computes the filter on the buffer image and then rank 0 gathers the results after they are all done to put the image back together. Rank 0 then saves the current image and copies it into a buffer so that the next filter can be applied where the same thing happens. This happens again for the third filter and rank 0 saves the final result before freeing all the memory that was used up.

Performance Experiments:

Tables:

Small Input:

Threads/Processes:	1	2	4	8
Serial Avg Time	0.1028325s	0.1028325s	0.1028325s	0.1028325s
OpenMP Avg Time	0.1031244s	0.0620209s	0.0416498s	0.0392459s
OpenMP Speedup	0.9971694	1.6580297	2.4689794	2.6202100
OpenMP Efficiency	0.9971694	0.8290148	0.6172448	0.3275262
MPI Avg Time	0.1002707s	0.0625040s	0.0457287s	0.0198383s
MPI Speedup	1.0255488	1.6452147	2.2487518	5.1835338
MPI Efficiency	1.0255488	0.8226073	0.5621879	0.6479417

Medium Input:

Threads/Processes:	1	2	4	8
Serial Avg Time	0.5828731s	0.5828731s	0.5828731s	0.5828731s
OpenMP Avg Time	0.5834890s	0.3149608s	0.2099036s	0.1380439s

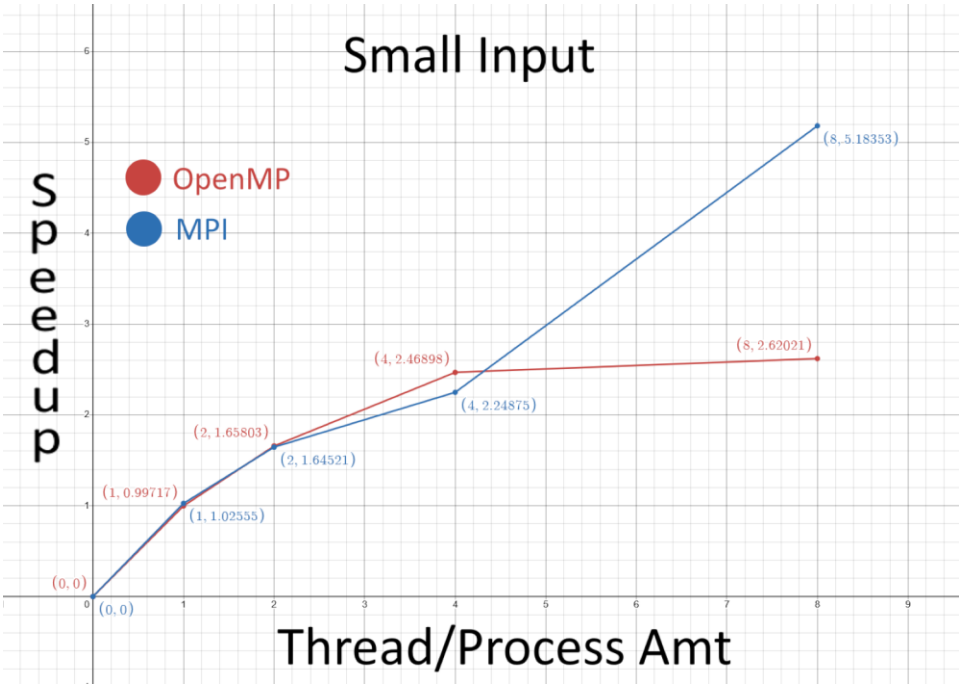
OpenMP Speedup	0.9989444	1.8506210	2.7768609	4.2223749
OpenMP Efficiency	0.9989444	0.9253105	0.6942152	0.5277968
MPI Avg Time	0.6353564s	0.3729889s	0.2797541s	0.2140722s
MPI Speedup	0.9173954	1.5627089	2.0835194	2.7227874
MPI Efficiency	0.9173954	0.7813544	0.5208798	0.3403484

Large Input:

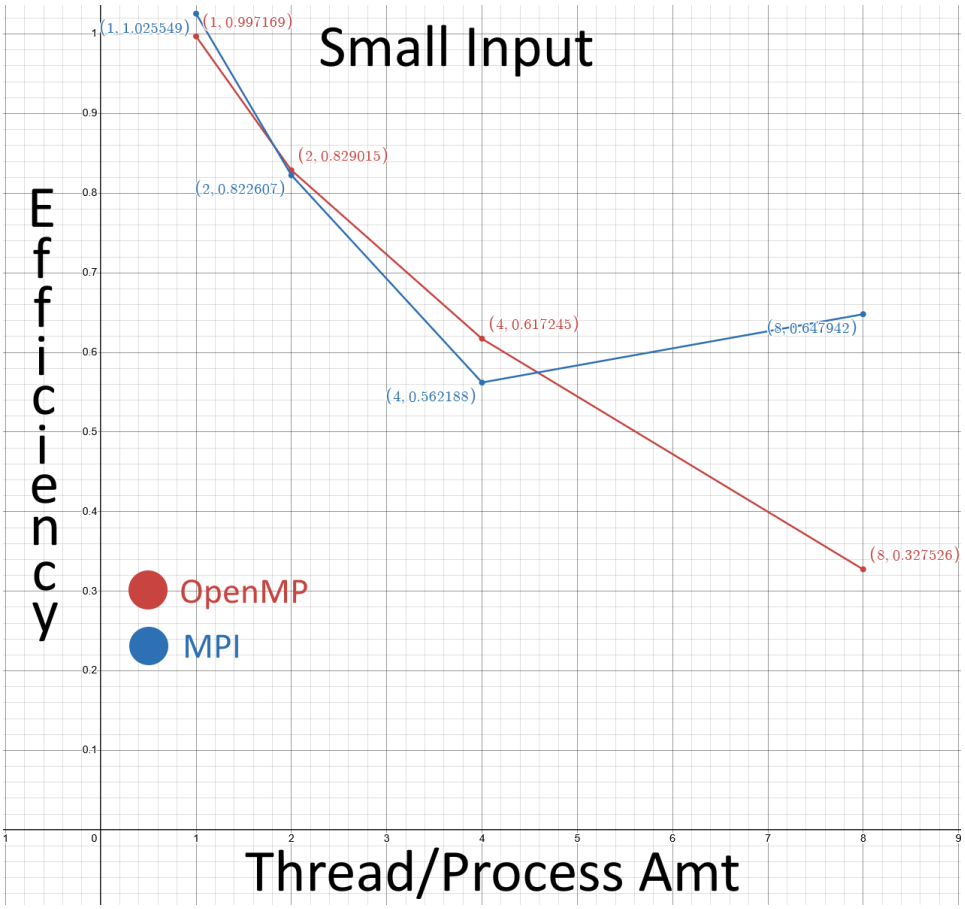
Threads/Processes:	1	2	4	8
Serial Avg Time	3.7278550s	3.7278550s	3.7278550s	3.7278550s
OpenMP Avg Time	3.5356358s	2.0897877s	1.2835509s	0.9067698s
OpenMP Speedup	1.0543662	1.7838438	2.9043296	4.1111371
OpenMP Efficiency	1.0543662	0.8919219	0.7260824	0.5138921
MPI Avg Time	4.0067553s	2.2126951s	1.3900621s	0.8652204s
MPI Speedup	0.9303924	1.6847576	2.6817902	4.3085611
MPI Efficiency	0.9303924	0.8423788	0.6704475	0.5385701

Plots:

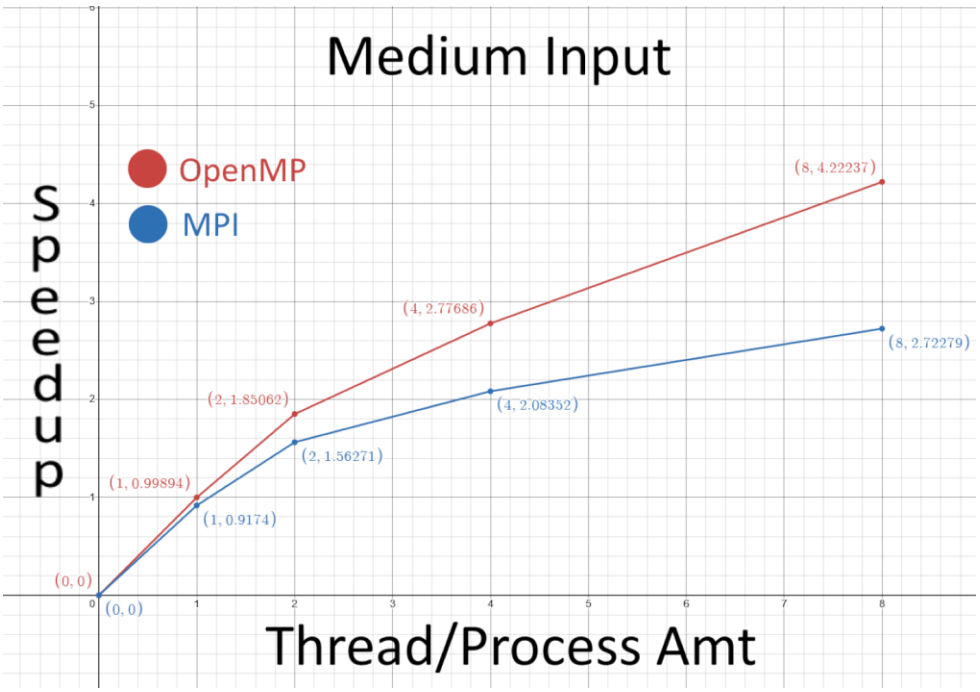
Small Input Speedup:



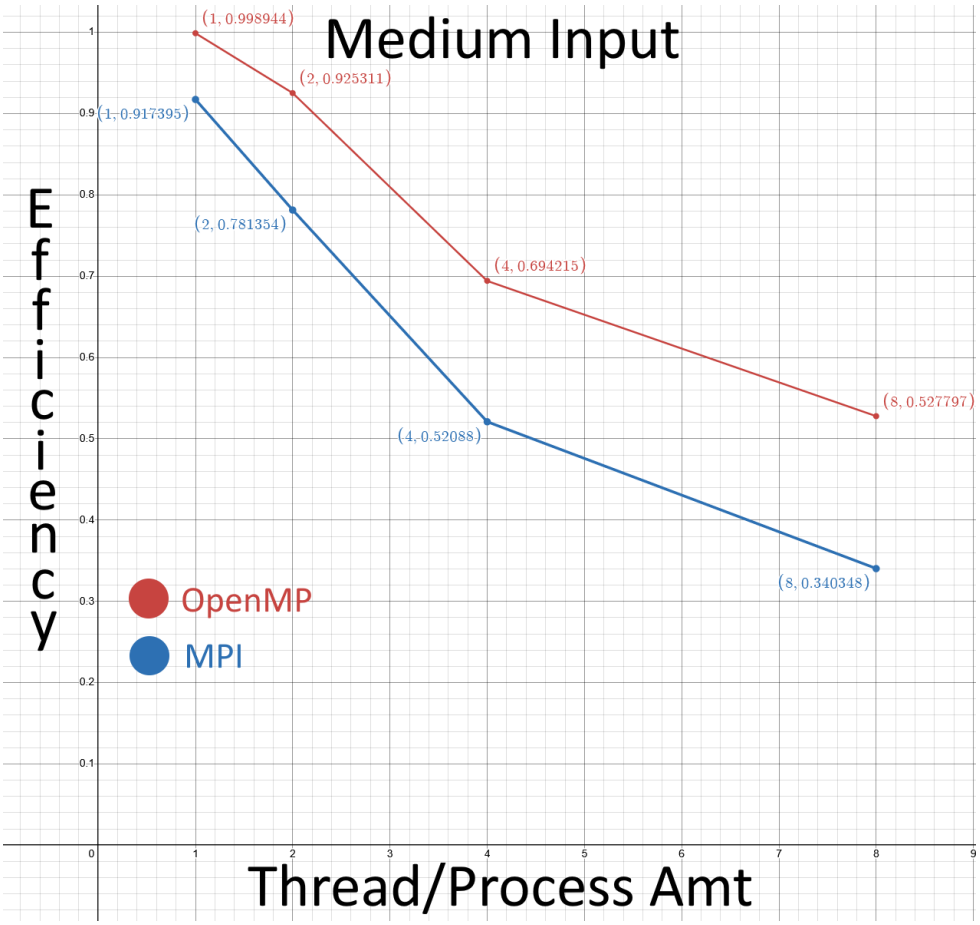
Small Input Efficiency:



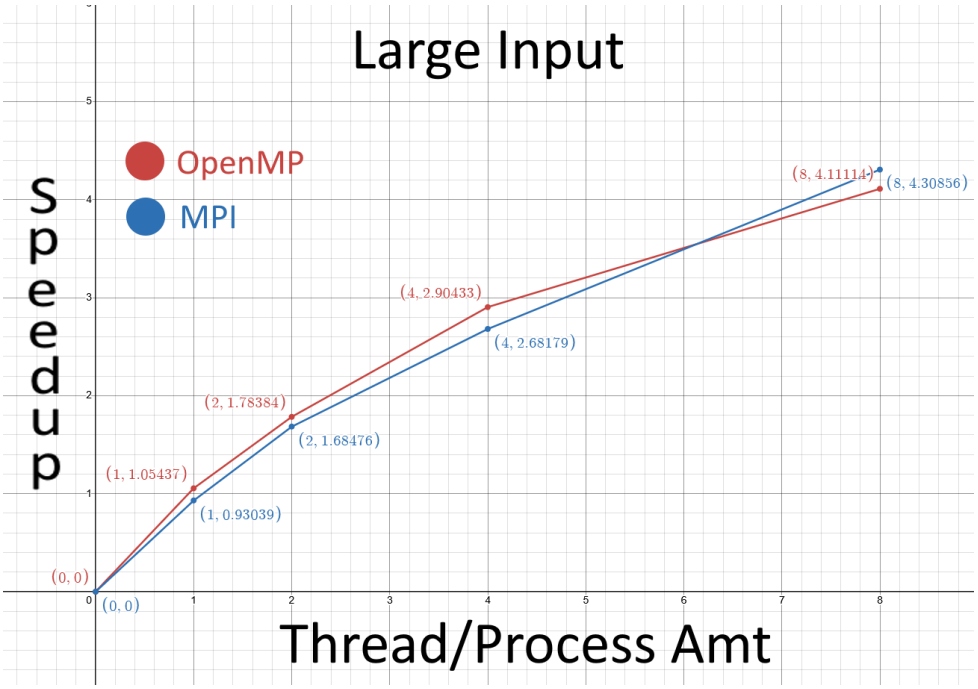
Medium Input Speedup:



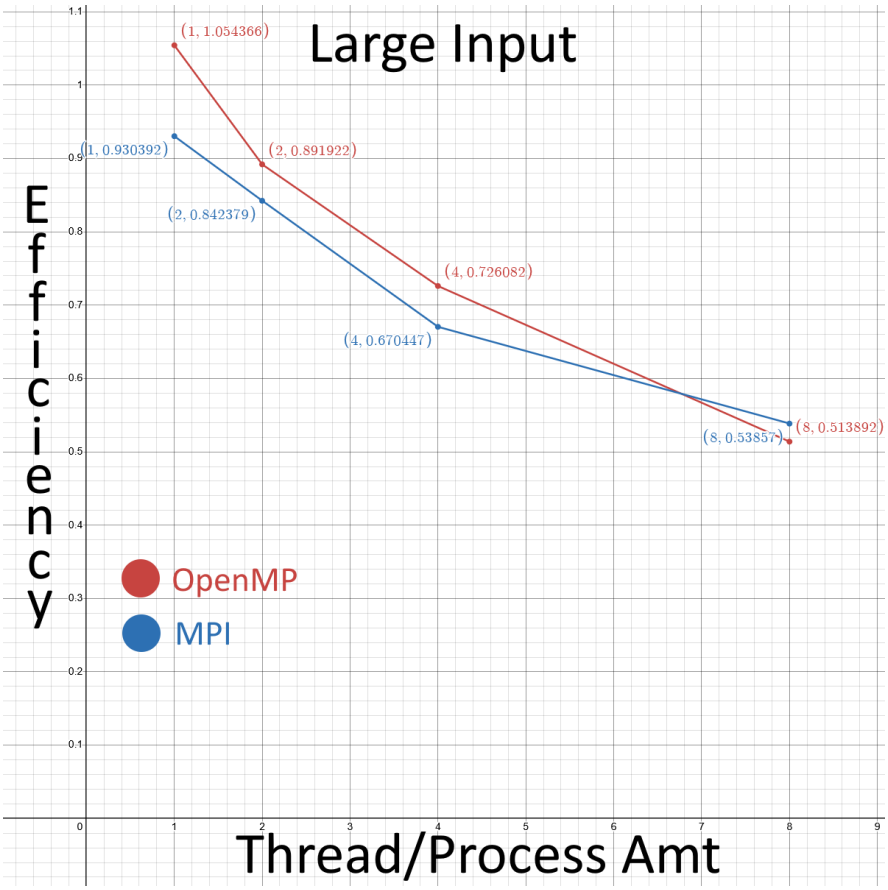
Medium Input Efficiency:



Large Input Speedup:



Large Input Efficiency:



Speedup, Efficiency, Bottleneck Discussion:

The results show that both OpenMP and MPI benefit more as the image size increases, because the larger workloads give more useful computation per thread or process. For the small input, the speedup curves rise at first but become less stable, especially for MPI at 8 processes, which likely reflects the very short runtime and higher sensitivity to overhead and measurement variation. For the medium input, OpenMP scales better, where it reachs 4.22× speedup and 0.53 efficiency at 8 threads, where with MPI's it's 2.72× speedup and 0.34 efficiency at 8 processes. This matches the medium-input graphs, where the OpenMP line stays above the MPI line. For the large input, both approaches improve further, and MPI slightly surpasses OpenMP at 8 workers, with 4.31× vs 4.11× speedup, while both efficiencies are close at about 0.54 and 0.51.

These result match the code design as in OpenMP, the main computation is the pixel by pixel work inside Gaussian, Median, and Sobel, while only timing, saving images, and

buffer swapping remain serial, making most of the work benefit directly from shared memory parallelism. In MPI, each stage must scatter rows, exchange halo data, and gather results back to rank 0, so communication and synchronization add overhead at every filter. That is why OpenMP performs better on the medium case, where communication cost is still significant, while MPI becomes more competitive on the large case, where the heavier computation better offsets that overhead. Overall, the main OpenMP bottlenecks are the remaining serial sections and shared memory bandwidth, while the main MPI bottlenecks are repeated communication, halo exchange, and rank-0 coordination.